MVP signifie Minimum Viable Product, en français « produit minimum viable ». C'est la plus petite version utilisable de ta plateforme, qui permet de vérifier que l'idée fonctionne avec de vrais utilisateurs. Il ne s'agit pas d'une version bâclée : elle fait peu de choses, mais elle les fait correctement.

Pourquoi c'est important pour toi

Le cahier des charges complet représente environ 14 mois-personnes. Seul, tu ne peux pas tout construire d'un coup. Plutôt que de viser la plateforme complète, tu livres d'abord le MVP, tu le fais tester, puis tu enrichis.

Ce que contient ton MVP (détail dans la section 4.2 du guide)

connexion sécurisée et import des comptes par fichier CSV ;

référentiels essentiels : UE, enseignants, inscriptions, délégués ;

évaluation anonyme complète : questionnaire de 19 critères, jeton, urne, seuil de 5 répondants ;

cahier de texte : saisie par le délégué, visa de l'enseignant, volumes horaires ;

résultats pour l'enseignant : fiche avec moyennes et mention de l'article 77 ;

synthèse affichée à l'écran pour le Chef de Département.

Ce qui est reporté en « version 2 »

Double authentification, exports Word et Excel, rappels automatiques, SMS, comparaisons entre départements, évolution des résultats entre semestres, etc.

Ce que ça te donne

Le MVP te permet de lancer un pilote dans un seul département, pendant une vraie campagne de fin de semestre. Les retours des étudiants et des enseignants te diront ensuite quoi ajouter en priorité.

Le sigle apparaît dans le guide sans définition. Veux-tu que j'ajoute « MVP » avec sa définition dans le glossaire du document Word ?

je n'ai pas vraiment compris comment implémenter l'étape 2 de mon plan

Je pars du principe que tu parles de l'Étape 2 de la section 8 du guide : « Créer la structure du dépôt ». C'est la seule étape qui demande de « l'implémenter » avec des commandes. Si tu pensais à une autre étape 2 (le plan de 6 semaines ou la section 6), dis-le-moi et je la détaille à la place.

Ce que tu construis

Tu crées un dossier unique, enp-evaluation, qui contient tout le projet : l'API (backend), le site (frontend), la documentation et les fichiers de déploiement. C'est plus simple à gérer seul que plusieurs dépôts. À la fin, tu auras :

```
enp-evaluation/
├── apps/
│   ├── api/        ← le backend (NestJS)
│   └── web/        ← le frontend (React + Vite)
├── docs/           ← documentation et journal des décisions
├── infra/          ← fichiers de déploiement (plus tard)
├── .github/workflows/
├── .env.example
├── .gitignore
└── README.md
```

Compte 1 à 2 heures. Sous Windows, utilise Git Bash (installé avec Git) plutôt que l'invite de

commandes, pour que les commandes ci-dessous fonctionnent.

Avant de commencer

Vérifie que ces commandes affichent un numéro de version (c'est l'objet de l'Étape 0) :

```
git --version
node --version
npm --version
```

Si node --version affiche un numéro inférieur à 20, installe Node 20 ou plus récent.

1. Créer le dépôt sur GitHub

Sur github.com, clique sur New repository.

Nom : enp-evaluation. Visibilité : Private.

Ne coche aucune option (ni README, ni .gitignore, ni licence). Un dépôt vide évite des conflits au premier envoi.

2. Créer le dossier et initialiser Git

```
mkdir enp-evaluation
cd enp-evaluation
git init -b main
mkdir apps docs infra
mkdir -p .github/workflows
```

3. Créer le backend (NestJS)

```
npm install -g @nestjs/cli
cd apps
nest new api --package-manager npm --skip-git
```

Le dossier api est créé et les dépendances s'installent (quelques minutes). L'option --skip-git est importante : sans elle, NestJS crée un deuxième dépôt Git dans api, ce qui casse ton dépôt principal.

Vérifie que ça marche :

```
cd api
npm run start:dev
```

Ouvre http://localhost:3000 dans ton navigateur : tu dois voir Hello World!. Arrête avec Ctrl + C.

4. Créer le frontend (React + Vite)

```
cd ..
npm create vite@latest web -- --template react-ts
cd web
npm install
npm run dev
```

Si l'outil pose des questions supplémentaires, garde les choix par défaut. Ouvre l'adresse affichée (en général http://localhost:5173) : tu dois voir la page d'accueil de Vite. Arrête avec Ctrl + C, puis reviens à la racine :

```
cd ../..
```

5. Créer les fichiers de la racine

```
touch .gitignore .env.example README.md docs/decisions.md
touch infra/.gitkeep .github/workflows/.gitkeep
```

Les .gitkeep sont des fichiers vides : Git ne conserve pas les dossiers vides, ceux-ci forcent donc la sauvegarde de infra et .github/workflows.

Contenu de .gitignore (fichiers à ne jamais envoyer sur GitHub) :

```
node_modules/
dist/
.env
*.log
infra/backups/
```

Contenu de .env.example (modèle sans aucune vraie valeur secrète) :

```
PORT=3000
DATABASE_URL=postgresql://enp:change-me@localhost:5432/enp_evaluation
JWT_ACCESS_SECRET=remplacer-par-une-longue-valeur-aleatoire
JWT_REFRESH_SECRET=remplacer-par-une-autre-valeur
BALLOT_TOKEN_SECRET=remplacer-par-un-secret-distinct
NODE_ENV=development
```

Dans README.md, écris pour l'instant le nom du projet et une phrase de description. Dans docs/decisions.md, note tes premiers choix (stack, hébergement au nom de l'ENP, architecture de l'anonymat). Ce journal t'évitera de remettre tes décisions en cause plus tard.

6. Premier commit et envoi sur GitHub

```
git add .
git commit -m "chore: initialise la structure du projet"
git remote add origin https://github.com/TON-COMPTE/enp-evaluation.git
git push -u origin main
```

Remplace TON-COMPTE par ton nom d'utilisateur GitHub. Git ouvre normalement une fenêtre de connexion GitHub dans le navigateur. Si l'envoi est refusé à cause du mot de passe, GitHub n'accepte plus les mots de passe pour cette opération : installe GitHub CLI et lance gh auth login.

7. Vérifier que l'étape est réussie

git status indique « nothing to commit, working tree clean ».

Le dépôt apparaît sur GitHub avec les dossiers apps, docs, infra et .github.

Le backend répond sur localhost:3000 et le frontend sur localhost:5173.

git ls-files | grep env affiche uniquement .env.example, jamais .env.

Problèmes fréquents

Message ou symptôme	Cause	Solution
nest: command not found	Le dossier des commandes npm n'est pas dans le PATH	Remplace par npx @nestjs/cli new api --package-manager npm --skip-git
Erreur de permission (EACCES) avec npm install -g	Droits insuffisants sur Linux ou Mac	Utilise npx @nestjs/cli ... sans installation globale
adding embedded git repository au moment du git add	Tu as oublié --skip-git	Supprime le faux dépôt : rm -rf apps/api/.git, puis refais git add .
Port 3000 is already in use	Un autre programme utilise ce port	Ferme-le, ou change PORT dans le fichier .env
git push rejeté (« fetch first »)	Le dépôt GitHub n'était pas vide	git pull origin main --allow-unrelated-histories, puis réessaie
Erreurs au démarrage de	Version trop ancienne	Installe Node 20 ou plus récent (avec

Message ou symptôme	Cause	Solution
Node		nvm install 20)

Une fois ces cases cochées, tu passes à l'Étape 3 : lancer PostgreSQL en local avec Docker. Si une commande échoue, copie-moi le message d'erreur exact et le nom de ton système (Windows, Mac ou Linux), et je te dis quoi corriger.

Ce n'est pas un problème : le terminal pose une question interactive. Il s'agit très probablement du choix du gestionnaire de paquets, c'est-à-dire l'outil qui télécharge les bibliothèques JavaScript.
Que choisir
Sélectionne npm. C'est celui qui est déjà installé avec Node.js et celui utilisé dans tout le guide.
Comment répondre
Utilise les flèches haut et bas du clavier pour placer le curseur sur npm.
Appuie sur Entrée.
Les autres choix (yarn, pnpm) sont des outils équivalents, mais ils ne sont pas installés chez toi. Si tu en choisis un par erreur, tu auras une erreur « command not found ».
Si le terminal affiche la question malgré l'option --package-manager npm, c'est que ta version de l'outil la redemande : ça ne change rien, réponds simplement npm.
Si c'est une autre question
Si le texte de la question est différent, voici les réponses selon l'étape :

Étape	Question	Réponse
Frontend (npm create vite)	Select a framework	React
Frontend (npm create vite)	Select a variant	TypeScript (pas « TypeScript + SWC »)
Frontend (npm create vite)	Use rolldown-vite / Install with npm and start now? (selon la version)	No pour l'option expérimentale, et No pour « start now » (tu lanceras npm install toi-même)

Une fois la réponse donnée, l'installation prend plusieurs minutes : des lignes défilent, c'est normal. Attends de revoir le $ avant de taper la commande suivante.
Si ce n'est ni l'un ni l'autre, copie-moi le texte exact affiché dans le terminal (question et choix proposés), et je te dirai quoi sélectionner.
Ton backend fonctionne : la ligne Nest application successfully started le confirme, et l'installation s'est bien terminée. L'erreur 401 à la fin ne vient pas de ton code.
Ce qui s'est passé
Le CLI a activé @nestjs/observe, le module de suivi (traçage, journaux, métriques) de NestJS, même si je t'avais conseillé de le refuser. Ce module essaie d'envoyer des données de suivi à un service externe. Comme tu n'as pas de compte ni de clés, il est refusé avec l'erreur 401 environ une minute après le démarrage.
Pourquoi je te conseille de le retirer
Il pollue les messages de ton terminal avec une erreur à chaque démarrage.
Il envoie des données de suivi vers un service tiers. Ta plateforme manipulera des données d'étudiants et un système d'anonymat : tu dois garder la maîtrise de ce qui sort du serveur,

comme prévu dans le guide.
Comment le retirer
1. Arrête le serveur. Clique dans le terminal et tape Ctrl + C.
2. Repère tous les endroits concernés. Depuis apps/api, tape :
grep -rni observe src package.json
Tu verras les lignes qui mentionnent observe, normalement dans src/app.module.ts et package.json.
3. Modifie src/app.module.ts. Ouvre-le dans VS Code (depuis la racine du projet, tape code ., puis ouvre apps/api/src/app.module.ts). Supprime :
la ligne d'import qui contient ObserveModule ;
l'entrée ObserveModule.forRoot(...) dans la liste imports.
Le fichier doit ressembler à ceci :
ts
import { Module } from '@nestjs/common';import { AppController } from './app.controller';import { AppService } from './app.service';@Module({ imports: [], controllers: [AppController], providers: [AppService],})export class AppModule {}
Ton fichier peut être un peu différent. Ce qui compte, c'est qu'il ne reste plus aucune mention de ObserveModule. S'il est plus complexe que cet exemple, colle-moi son contenu et je te dirai quoi retirer.
4. Désinstalle le paquet. Le résultat du grep t'indique son nom exact. S'il s'agit de @nestjs/observe :
npm uninstall @nestjs/observe
5. Vérifie. Relance avec npm run start. Le journal ne doit plus afficher ObserveModule ni ObserveAgentWorker. Attends une minute et demie pour confirmer que l'erreur 401 ne revient pas. Ouvre ensuite http://localhost:3000 : tu dois voir Hello World!. Arrête avec Ctrl + C.
Pour la suite
Utilise plutôt npm run start:dev pendant le développement : le serveur redémarre tout seul quand tu modifies un fichier. La suite de l'étape 2 est la création du frontend (React + Vite), puis des fichiers de la racine et le premier commit.
Si l'une de ces commandes affiche une erreur, copie-moi le message et le contenu de app.module.ts.
ce n'est pas moi qui l'activé, c'était automatique
Clarification d'un malentendu sur l'origine de l'activation.
Tu as raison, ce n'est pas toi. Ma phrase laissait entendre le contraire. Dans ton affichage, le CLI n'a posé aucune question sur l'observabilité : il a installé @nestjs/observe automatiquement. Mon conseil de répondre « No » supposait qu'une question apparaîtrait, ce qui n'a pas été le cas chez toi. Tu n'as rien fait de travers.
Le plus simple : recréer le projet sans ce module
Tu n'as encore rien écrit dans api, donc recréer le projet est plus rapide que de modifier des fichiers à la main. Le CLI propose une option pour refuser le module dès le départ (--no-observe, d'après les notes de version du CLI, mais je ne l'ai pas testée).
Arrête le serveur s'il tourne encore : Ctrl + C dans le terminal.
Remonte d'un dossier : tu dois être dans apps, pas dans apps/api.
cd ..

Supprime l’ancien projet et recrée-le :

rm -rf api

nest new api --package-manager npm --skip-git --type cjs --no-observe

Vérifie, une fois l’installation terminée :

cd api

npm run start:dev

Le journal ne doit plus mentionner ObserveModule ni ObserveAgentWorker, et http://localhost:3000 doit afficher Hello World!. Si l’erreur 401 ne revient pas après une minute et demie, c’est réglé.

Si l’option --no-observe est refusée

Si le terminal répond que l’option est inconnue, utilise la méthode manuelle de ma réponse précédente : supprimer ObserveModule dans src/app.module.ts, puis npm uninstall @nestjs/observe. Dans ce cas, colle-moi le message d’erreur ou le contenu de app.module.ts et je t’indique les lignes exactes.

Pour la suite

Cette découverte vaut pour le guide : à l’étape 2, la commande de création du backend devrait inclure --type cjs --no-observe. Je mettrai le guide Word à jour si tu veux.

posko@JAK_DESKTOP MINGW64 ~/enp-evaluation/apps (main)

$ nest new api --package-manager npm --skip-git --type cjs --no-observe

error: unknown option '--type'

Je me suis trompé : j’ai tiré --type cjs des notes de version du schéma interne de NestJS, mais la commande nest new ne l’accepte pas. C’est la seule option refusée dans ton message, donc le reste de la commande est probablement bon.

Relance sans --type

Tu choisiras CJS à la main, comme tout à l’heure. Depuis le dossier apps, tape :

nest new api --package-manager npm --skip-git --no-observe

Quand la question sur le système de modules apparaît, place le curseur sur CJS (la ligne mentionne Jest) avec les flèches, puis appuie sur Entrée.

Attends la fin de l’installation, jusqu’à revoir le $.

Vérifier que le module de suivi est bien absent

Reste dans apps et tape :

grep -rni observe api/src api/package.json

Si la commande n’affiche rien, @nestjs/observe n’est pas installé : c’est réglé. Passe ensuite au test habituel avec cd api puis npm run start:dev. Le journal ne doit plus mentionner ObserveModule, et http://localhost:3000 doit afficher Hello World!.

Si une erreur s’affiche

unknown option '--no-observe' : retire cette option et relance sans elle :

nest new api --package-manager npm --skip-git

Choisis CJS, puis retire le module de suivi à la main en suivant les étapes 2 à 5 de ma réponse précédente : src/app.module.ts, puis npm uninstall @nestjs/observe.

Le dossier api existe déjà : supprime-le d’abord avec rm -rf api, puis relance la commande.

Pour voir la liste exacte des options que ta version du CLI accepte, tape nest new --help. Si l’une de ces commandes affiche une nouvelle erreur, copie-moi le texte tel quel.

17:

Cette fois, c'est bon. Le journal ne mentionne plus ObserveModule ni ObserveAgentWorker, et il indique Found 0 errors et Nest application successfully started. Le module de suivi n'est donc plus installé. Il ne manque que la confirmation dans le navigateur.

1. Vérifier le backend

Ouvre http://localhost:3000 : tu dois voir Hello World!.

Laisse tourner une minute et demie. L'erreur 401 d'avant ne doit pas réapparaître.

Clique dans le terminal et tape Ctrl + C pour arrêter le serveur.

2. Créer le frontend (React + Vite)

Tu es dans apps/api. Remonte d'un dossier, puis crée le frontend :

cd ..

npm create vite@latest web -- --template react-ts

Si l'outil pose des questions, garde les choix par défaut et refuse les options expérimentales ou « démarrer maintenant » :

Question	Réponse
Option expérimentale (par exemple « rolldown »)	No
Install with npm and start now?	No (tu installeras toi-même)

Puis :

cd web

npm install

npm run dev

Ouvre l'adresse affichée (en général http://localhost:5173) : tu dois voir la page d'accueil de Vite.

Arrête avec Ctrl + C, puis reviens à la racine du projet :

cd ../..

pwd

Le chemin affiché doit se terminer par enp-evaluation.

3. Ce qui reste de l'étape 2

créer les fichiers de la racine : .gitignore, .env.example, README.md, docs/decisions.md ;

faire le premier commit et l'envoyer sur GitHub.

Dis-moi quand tu as vu la page de Vite et je te guide pour ces fichiers. Si une erreur s'affiche, copie-moi le texte tel quel.

Une dernière remarque : la commande de création du backend dans ton guide Word est désormais fausse (le CLI demande CJS ou ESM, et active le module de suivi par défaut). Souhaites-tu que je mette à jour cette étape dans le guide avec la commande que tu viens d'utiliser ?

Dans README.md, écris pour l'instant le nom du projet et une phrase de description. Dans docs/decisions.md, note tes premiers choix (stack, hébergement au nom de l'ENP, architecture de l'anonymat). Ce journal t'évitera de remettre tes décis

comment ?

Ces deux fichiers sont des fichiers texte à ouvrir dans VS Code, à remplir et à enregistrer. Rien à taper dans le terminal, sauf pour les ouvrir.

1. Ouvrir les fichiers

Dans Git Bash, place-toi à la racine du projet (le chemin doit se terminer par enp-evaluation) :

pwd

ls -a

ls docs
Tu dois voir README.md à la racine et decisions.md dans docs. Si l'un manque, crée-le :
touch README.md docs/decisions.md
Ouvre ensuite le projet dans VS Code :
code .
Si code n'est pas reconnu, ouvre VS Code, puis Fichier → Ouvrir le dossier et choisis enp-evaluation. Dans la colonne de gauche, clique sur README.md pour l'ouvrir.
2. Remplir README.md
C'est la page d'accueil du projet sur GitHub. Colle ce texte dans le fichier, puis enregistre avec Ctrl + S :
Plateforme d'évaluation des enseignements et cahier de texte (ENP)

Plateforme web permettant aux étudiants d'évaluer anonymement les enseignements
(article 77 du Règlement des Études de l'ENP), aux délégués de renseigner le
cahier de texte et aux enseignants de consulter leurs résultats.

Statut

Projet en cours de démarrage (MVP en préparation).

Structure du dépôt

- `apps/api` : backend (NestJS)
- `apps/web` : frontend (React + Vite)
- `docs` : documentation et journal des décisions
- `infra` : fichiers de déploiement

Lancer en local

Backend : `cd apps/api` puis `npm run start:dev` (http://localhost:3000)
Frontend : `cd apps/web` puis `npm run dev` (http://localhost:5173)

Documentation

Voir `docs/decisions.md` pour les choix techniques et leurs raisons.
Le fichier utilise le format Markdown : # crée un titre, - une liste, les accents inverses (`) mettent du texte en police de code. GitHub l'affiche joliment sur la page du dépôt.
3. Remplir docs/decisions.md
C'est ton journal des décisions : pour chaque choix important, tu notes la décision et sa raison. Dans quelques mois, tu sauras pourquoi tu as choisi ceci plutôt que cela. Ouvre docs/decisions.md, colle ce texte (adapte si tu changes d'avis) et enregistre :
Journal des décisions

2026-10-04 — Architecture : monolithe modulaire

- Décision : un seul backend NestJS découpé en modules, une seule base PostgreSQL.
- Raison : projet mené seul, charge faible en moyenne ; les microservices ajouteraient de la complexité inutile.

2026-10-04 — Technologies
- Frontend : React + TypeScript + Vite.
- Backend : NestJS, système de modules CJS (CommonJS) avec Jest.
- Base de données : PostgreSQL, lancée avec Docker Compose en local.
- Raison pour CJS : plus de documentation et d'exemples, et le code proposé par les assistants d'IA y est plus souvent compatible.

2026-10-04 — Pas de module de suivi externe (@nestjs/observe)
- Décision : ne pas utiliser @nestjs/observe, installé automatiquement par le générateur de projet puis retiré.
- Raison : il envoie des données de suivi vers un service tiers ; la plateforme doit garder la maîtrise de ce qui sort du serveur.

2026-10-04 — Anonymat des évaluations
- Décision : deux espaces de stockage séparés, sans clé commune.
  - Registre de participation : qui a voté (sans la réponse).
  - Urne : ce qui a été voté (sans l'identité).
- Un jeton à usage unique, sans identité, autorise le dépôt dans l'urne.
- Rôles de base de données distincts pour l'écriture dans chaque espace.
- Résultats affichés seulement à partir de 5 répondants et après la clôture.
- Raison : l'anonymat doit être vérifiable, pas seulement promis.

2026-10-04 — Périmètre du MVP
- Inclus : comptes et imports CSV, évaluation anonyme, cahier de texte, résultats enseignant, synthèse à l'écran.
- Reporté en version 2 : double authentification, exports Word et Excel, rappels automatiques, SMS, comparaisons entre semestres.

2026-10-04 — Hébergement
- Décision : serveur et domaine ouverts au nom de l'ENP (crédits du pack étudiant utilisés pendant la phase de test).
- Raison : la plateforme doit survivre à mon statut d'étudiant.
- À confirmer avec l'ENP.

2026-10-04 — Outils d'IA
- Figma : maquettes. Bolt.new : écrans de l'interface. Jules : tâches ciblées, dépôt privé, relecture de chaque pull request.
- Anonymat, authentification et droits d'accès : écrits par moi, avec relecture.
- Aucune vraie donnée d'étudiant dans les outils d'IA.

À valider avec l'ENP
- Référent à la Scolarité et accord écrit de la Direction.
- Authentification des étudiants (matricule + mot de passe).
- Données de test disponibles (UE, inscriptions, affectations).
- Département pilote et date de la campagne.

La date 2026-10-04 est celle d'aujourd'hui. Pour toute nouvelle décision, ajoute une section en bas avec sa date.

4. Vérifier que c'est enregistré

Dans VS Code, un point blanc sur l'onglet du fichier signifie qu'il n'est pas enregistré : fais Ctrl + S.

Dans Git Bash, à la racine :

git status

README.md et docs/decisions.md doivent apparaître. Ils seront envoyés au premier commit.

Il te reste à compléter .gitignore et .env.example (leur contenu figure dans ma réponse sur l'étape 2), puis à faire le commit. Veux-tu que je te guide pour ces deux fichiers ?

Cahier_des_Charges_Plateforme_Evaluation_ENP_(review par Dr ATTIPOU).docx

DOCX

/compact

En attendant de passer à l'étape 6.6, je veux que tu sache que le cahier de charge a été amélioré (je vais te l'envoyer). L'autre chose est de savoir que l'ENP a beaucoup de site par exemple ENP-CL (ENP Campus de Lomé) et ENP-CK (ENP campus de Kara) et ensuite plusieurs départements par campus qui ne sont pas forcément les mêmes sur les deux campus avec leur chef respectif.

Donc on doit adapter le projet.

Adaptation du cahier des charges à un contexte multi-campus.

Adaptation du cahier des charges à un contexte multi-campus.

Je commence par lire le cahier des charges révisé pour voir ce que Dr ATTIPOU a modifié, avant de proposer l'adaptation multi-campus.

Lu un fichier, a exécuté une commande

Continuer

A exécuté une commande

Continuer

Exécuté 3 commandes

Continuer

Listant les points à trancher et esquissant le plan révisé.

J'ai lu la version révisée, et l'organisation multi-campus se gère bien. Rien n'est à jeter dans ce que tu as construit, mais plusieurs droits d'accès sont à reprendre. Je te donne l'analyse et le plan, et je te fournirai l'étape de code après tes réponses, parce que la Scolarité et la Direction des Études conditionnent la conception.

1. Ce que Dr ATTIPOU a modifié

Le document contient 67 insertions et 3 suppressions suivies. L'essentiel :

Un nouvel acteur, la Direction des Études, qui contrôle l'application des règles sur tous les campus. Elle regroupe les résultats, fait la synthèse et publie les statistiques au niveau de l'ENP. Elle a sa colonne dans la matrice des droits : résultats, commentaires, grille de synthèse, et « Générer les stats ».

Le module Pilotage, les écrans et la formation incluent désormais la Direction des Études.
Des définitions plus claires pour API et PWA.
Le critère B2 reformulé : « …accessible en dehors des séances aux horaires indiqués sur le syllabus ».

2. Quatre points à corriger ou à faire valider

B2 n'est plus la grille officielle. L'Annexe A dit reproduire la grille à l'identique, et l'article 77 s'y réfère. Tant que l'ENP n'a pas officialisé cette formulation, je garde la version 1 du questionnaire en base. Je préparerai une version 2 pour les nouvelles campagnes, ce qui est possible puisque chaque campagne fige sa version.

PWA est mal défini. Il signifie Progressive Web App, pas « Portable Web Application ». À corriger dans le glossaire.

La Direction des Études n'est pas alignée avec la règle RG-08, qui limite la communication des résultats à l'enseignant, au Chef de Département et à la Scolarité. L'article 77 exige aussi que les résultats ne soient pas le seul élément d'appréciation d'un enseignant. Cela justifie des statistiques par campus, département et UE, avec le seuil de 5 répondants, et pas de classement nominatif des enseignants entre campus.

Il manque des exigences. Le module Pilotage n'a aucune ligne pour les statistiques ni pour la consolidation. Je propose trois exigences : consolidation multi-campus, statistiques globales avec seuil, exports.

3. Ce qui fonctionne déjà, et ce qui manque

Ton schéma gère déjà plusieurs campus : ref.department est rattaché à un campus, et une campagne porte sur un département, donc sur un campus. Deux départements du même nom sur deux campus sont déjà deux départements distincts, avec deux campagnes, deux synthèses et deux chefs. La question 12 du cahier prévoyait d'ailleurs une plateforme unique avec données cloisonnées.

Ce qui manque, ce sont les droits. Aujourd'hui, un utilisateur scolarite agit sur tous les campus, aucun chef n'est rattaché à un département, et le rôle Direction des Études n'existe pas.

4. Modèle proposé : la portée est une donnée

Rôle	Portée
Étudiant, délégué	Son campus, par son inscription (programme → département → campus)
Enseignant	Ses UE, par ses affectations ; il peut enseigner sur plusieurs campus
Chef de Département	Un département précis (donc un campus)
Scolarité	Un campus, ou tous (service central) selon la ligne d'attribution
Direction des Études	Tous les campus, en lecture et statistiques
Administrateur technique	Aucune donnée pédagogique

La portée sera stockée sur chaque ligne de rôle, pas écrite dans le code. Si la Scolarité est centrale ou par campus, seule la donnée change.

5. Ce que ça change

Élément	Changement

Élément	Changement
Base de données	Codes ENP-CL et ENP-CK ; code de département ; rôle direction_etudes ; portée sur les rôles
Campagnes	Chaque Scolarité n'agit que sur son périmètre ; la Direction des Études ne fait que consulter
Résultats	Chef de Département : son département ; Scolarité : son campus ; Direction : tous. L'administrateur technique perd l'accès (la matrice révisée le prévoit)
Modération, restitution	Limitées au campus de la campagne
Imports (6.6)	Une colonne campus (par exemple ENP-CL) ; un import ne peut viser que le périmètre de l'utilisateur
Frontend	Colonne campus, filtre selon le périmètre, menu de la Direction des Études ; Prompts F et E à mettre à jour
Tests	Nouveaux tests de cloisonnement : un chef de Kara ne voit rien de Lomé

Quelques tests existants seront à adapter, car les services reçoivent le périmètre en plus.

6. Plan révisé

Étape	Contenu
6.5b (maintenant)	Portées campus et département, chef rattaché à son département, rôle Direction des Études, cloisonnement des routes existantes
6.6	Référentiels filtrés par périmètre, imports avec codes de campus, tableau de bord par périmètre
6.7	Statistiques consolidées de l'ENP (Direction des Études) et exports
Ensuite	Premier déploiement, cahier de texte (la partie « chef de département » est déjà traitée), pilotage

7. Six points à trancher

Pour chacun, la recommandation s'applique si tu réponds « d'accord ».

Scolarité : un service central ou un par campus ? Reco : le modèle gère les deux, donc dis-moi qui ouvre les campagnes à Kara.

Chef de Département : un titulaire, avec ou sans intérimaire ? Reco : plusieurs lignes autorisées par département, la Scolarité gère l'attribution.

Direction des Études : accès au détail d'une UE et aux commentaires, ou seulement aux statistiques agrégées ? Reco : agrégats par campus, département et UE avec seuil de 5, détail seulement si l'ENP le confirme.

Questionnaire : le B2 révisé remplace-t-il la grille officielle ? Reco : version 1 jusqu'à validation officielle, version 2 prête.

Questionnaire unique pour tous les campus et cursus ? Reco : oui.

Calendrier : les campus ont-ils les mêmes dates de semestre ? Reco : dates propres à chaque campagne, déjà prévu.

8. Ce qu'il me faut

Envoie-moi, avec des noms fictifs si tu préfères, la liste code campus, nom, départements de chaque campus (par exemple ENP-CL, ENP-CK), sans noms de personnes. Voici ce que contient ta

base en ce moment :

docker exec enp-db psql -U enp -d enp_evaluation -c "SELECT c.id, c.nom, d.id AS dept, d.nom AS departement FROM ref.campus c LEFT JOIN ref.department d ON d.campus_id = c.id ORDER BY 1, 3;"

docker exec enp-db psql -U enp -d enp_evaluation -c "SELECT role, count(*) FROM identity.user_roles GROUP BY role ORDER BY 1;"

Dès que tu me donnes les réponses (« d'accord » pour toutes suffit), je te fournis l'étape 6.5b. Je peux aussi rédiger un addendum Word d'environ 4 pages à faire relire par Dr ATTIPOU : Direction des Études, organisation multi-campus, nouvelles exigences et point B2. Dis-moi si tu le veux.